

Verifier-Gated LLM NetOps: A Preregistered, Artifact-Backed Evaluation of Input-Sanitization and Deterministic Verification Against Adversarial Intent

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a preregistered paired confirmatory experiment (study_id cand-2026-03) that measured whether template-based input sanitization combined with deterministic verifier-gating (and at-most-one automated repair) changes the rate at which a deterministic validator accepts LLM-generated CLI configurations under adversarially mutated intents. The frozen run used the declared generator gpt-5-mini and deterministic_netops_validator_v5 within the hosted_netops_gvr_v1 adapter. Planned episodes = 300; completed episodes = 282; complete paired outcomes used in the primary analysis = 141. Observed per-pair acceptance: Baseline 135/141 (95.7447%); Guarded 141/141 (100.0%); paired absolute difference = 0.04255319148936165; exact two-sided McNemar $p = 0.03125$; 95% bootstrap percentile CI (10,000 resamples, seed = 1729) = [0.014184397163120567, 0.07801418439716312]. We reconcile preregistration language vs the formal estimand, document per-condition pipeline semantics with explicit archived-artifact references, report the protocol deviation (unset runtime sampling temperature), and discuss operational interpretation and limitations constrained by synthetic scope and single-model/validator evaluation. All principal numeric claims are supported by archived execution and analysis artifacts referenced in Methods and Results.

I. INTRODUCTION

Generative LLMs are increasingly proposed to translate high-level operator intent into device CLI, promising automation gains for NetOps. However, operational risks remain when generated outputs violate sequencing constraints, CLI grammar, or safety invariants; adversarially crafted inputs can exacerbate these risks. Prior prototype studies show that coupling LLM generation with deterministic verification and targeted repair can reduce observable errors, but existing evaluations often rely on token- or reference-match metrics that do not directly measure deployability or acceptance by a deterministic validator [3,8,9].

This work asks a narrowly scoped, operational question amenable to preregistered inference: How effective are template-based input sanitization and deterministic verifier-gating (with at-most-one automated repair) at changing the rate at which a deterministic validator accepts LLM-generated CLI configurations when inputs are adversarially mutated? The study cand-2026-03 preregistered a single formal estimand (paired_success_rate_difference_guarded_minus_baseline) and a confirmatory paired test. Crucially, because the

registered generation semantics were shared_initial_candidate, the paired comparison isolates downstream guarded-pipeline effects (sanitization, gating, permitted repair, and final validation) applied to a single shared LLM draft per intent rather than differences in initial generation.

Contributions. (1) A preregistered, artifact-backed paired experiment executed end-to-end and reconciled against the archived preregistration; (2) a precise, artifact-grounded description of per-condition pipeline semantics that demonstrates shared_initial_candidate reuse; (3) quantitative confirmatory results showing a small but statistically detectable absolute change in deterministic-validator acceptance under the guarded pipeline on a synthetic adversarial-intent bank; and (4) an explicit reconciliation of preregistration natural-language hypothesis wording with the formal estimand and a disclosure of a protocol deviation (unset sampling temperature) together with reproducibility guidance.

II. RELATED WORK

Deterministic network-verification tools. Formal, deterministic validators such as Header Space Analysis and VeriFlow (and related NetPlumber-style approaches) are widely used to encode and rapidly check forwarding, access-control, and sequencing invariants before deployment [5–7]. These tools operate over explicit, analyzable models of packet headers or configuration effects, and therefore directly report property-level violations (reachability, isolation, rule precedence) rather than syntactic or token-level differences. That representational difference explains their suitability as deployment gates in LLM-assisted NetOps: validators can accept or reject a candidate configuration with a semantics-focused criterion, but they require explicit formalization of the intended invariants and do not automatically cover every dynamic or protocol-level safety property. Prior literature therefore treats deterministic validators as precise but partial oracles whose effectiveness depends on the fidelity of the encoded specifications and the scope of checked properties [5–7,14].

LLM-assisted configuration and evaluation gaps. Recent empirical work that studies LLMs for NetOps (e.g., NetConEval, NetLLMBench, and zero-touch proposals) consistently finds that model-only generation produces both syntactic errors

and higher-level semantic mistakes (ordering, precondition violations, protected-interface breaches) when translating intent to CLI [3,8,9]. Many published evaluations focus on string- or reference-match metrics that measure surface similarity to a gold command sequence; such metrics correlate poorly with deployability in practice because they neither verify execution-order constraints nor check reachability/forwarding invariants. Surveys of AIOps/telecommunications emphasize this fragmentation in evaluation practices and call for operationally grounded benchmarks that measure deployability, safety, and validator acceptance rather than token overlap alone [4,9].

Generate-validate-repair architectures and tool-assisted orchestration. A common mitigation pattern is the generate-validate-repair architecture: an LLM produces a candidate, an automated verifier checks it, and feedback is converted into constrained repair prompts or corrective actions that produce revised candidates. Prototype studies and orchestration patterns (including Toolformer-style approaches) demonstrate that integrating external checks into the generation loop improves concrete outcome metrics on narrow tasks, provided verifier feedback is sufficiently precise and repair scope is constrained [11,3]. These demonstrations show methodological promise but also reveal dependencies: (a) repair effectiveness depends on the granularity and clarity of verifier diagnostics (e.g., whether the checker returns a parsable, actionable violation code vs a high-level failure label), and (b) bounded repair policies (limited number of automatic repair attempts and explicit constraints on allowable edits) are often necessary to avoid uncontrolled changes that could mask adversarial manipulations.

Security and adversarial inputs in LLM-integrated NetOps. The LLM-systems security literature documents prompt-injection and indirect prompt-attacks that can subvert downstream behavior when external or user-supplied text is trusted by an orchestrator [10]. These attack modes motivate operational defenses used in practice, such as template-based input sanitization, slot validation, and gating designs that deterministically block candidates failing explicit safety checks. Prior defensive evaluations are often performed on synthetic or constrained datasets and may not exhaustively explore multi-source or retrieval-augmented pipelines, leaving an open need for NetOps-specific threat-modeling and rigorous mitigation studies.

Benchmarks, measurement, and operational desiderata. Several benchmark efforts and proposals (NetConfEval, NetLLM-Bench, and related task suites) attempt to exercise common NetOps intents but differ in goals: some emphasize generation diversity, others target protocol-specific behaviors. Across these efforts a recurring limitation is the lack of standardized, validator-centered deployability metrics. This paper situates itself against those gaps by preregistering an estimand that uses deterministic-validator acceptance as the primary quantity of interest and by explicitly using `shared_initial_candidate` semantics so that post-generation pipeline elements (sanitization, gating, bounded repair) are the causal factors under test. Unlike many prior demonstrations, the present study em-

phasizes artifact-backed traceability (shared-generation files, per-episode validator traces, and provider-call accounting) to connect observed numeric differences to concrete pipeline actions and to diagnose discordant cases traceable to permitted repairs.

Synthesis and remaining limitations in prior work. In sum, prior work establishes three relevant points: (1) deterministic validators provide semantically meaningful acceptance criteria but require explicit property specification; (2) generate-validate-repair architectures can reduce observable errors but their safety depends on verifier diagnostic quality, repair scope, and gating policies; and (3) existing evaluations often use token-level metrics or nonstandardized benchmarks that do not directly measure deployability under an explicit danger model. These findings motivate the present preregistered, artifact-backed paired experiment that isolates post-generation safeguards under `shared_initial_candidate` semantics and records per-episode validator traces to support interpretive diagnostics.

III. METHODOLOGY

Overview and preregistration. The study was preregistered as `cand-2026-03` before observing outcomes (preregistration_sha256: `ba6ed25e84c7240fbe94f2d50b3e778fbfffd810ff2db632df8946e85f0f1373`). The preregistration specifies a paired confirmatory design (`shared_initial_candidate` generation semantics), a single primary estimand (`paired_success_rate_difference_guarded_minus_baseline`), and concrete analysis procedures (exact two-sided McNemar for the paired comparison; bootstrap-percentile 95% CIs with 10,000 resamples and seed = 1729). These analysis parameters and bootstrap metadata are archived (`analysis/analysis_log.jsonl`).

Task bank, adversary model, and scope. Tasks were produced by `deterministic_netops_task_generator_v5`; each task encodes: (i) an initial device-state snapshot, (ii) an operator intent template (intent-to-configuration), and (iii) a required command sequence and per-field constraints (e.g., `must-shutdown-before-MTU-change`). The synthetic bank exercises admin, MTU, and VLAN updates and a small set of workflow policies (protected interfaces, group-activity minima, must-be-down constraints). The adversary model used deterministic mutation heuristics applied to these templates (string-level mutations, slot-overrides, token insertions) as archived in `execution/task_manifest.jsonl` (see representative entries such as `execution/task_manifest.jsonl -> task-000001` payload). Scope was intentionally limited to these synthetic templates and the deterministic CLI grammar encoded in the validator; no production device logs or vendor-specific private data were used.

Orchestration, declared models, and protocol deviation. The orchestration adapter was `hosted_netops_gvr_v1` and the declared generator was `gpt-5-mini` (`execution/model_configuration.json`). The deterministic validator was `deterministic_netops_validator_v5`; validator outputs include `explicit_validation_before` and `validation_after` objects with parsed command counts, touched-field counts,

enumerated `violation_codes`, and `final_state` snapshots (examples: `execution/scoring/task-000001-baseline.json` and `execution/scoring/task-000001-guarded.json`). The preregistered pipeline constrained generation semantics to `shared_initial_candidate` (`independent_condition_generation = false`). The archived `model_configuration.json` records `declared_model_version = "gpt-5-mini"` and `temperature = null`; we disclose this unset temperature as a protocol deviation because provider-side defaults may influence sampling. To preserve internal comparability despite the unset parameter the pipeline used a single shared generation per pair (evidence: `analysis/deterministic_reconciliation.json`: `stage_counts.shared_initial_generation = 150`) and provider-call accounting (`hosted_model_call_event_count = 156`, `completed = 147`, `failed = 9`) corroborates the actual call pattern.

Per-pair timeline and concrete artifact evidence (`shared_initial_candidate` semantics). The registered `shared_initial_candidate` semantics produced the following deterministic per-pair timeline; each step and its archived artifact(s) are listed to permit exact forensic inspection: 1) Task instantiation and adversarial mutation: recorded in `execution/task_manifest.jsonl` (task payload contains the sanitized-orig intent and any slot changes). Representative entry: `task-000001` payload (`execution/model_manifest` and `execution/task_manifest.jsonl`). 2) Shared initial generation (single LLM call per pair): output saved to `execution/responses/task-<id>-shared-initial.txt` (e.g., `execution/responses/task-000001-shared-initial.txt`). Provider-stage accounting records `shared_initial_generation = 150` (`analysis/deterministic_reconciliation.json`). 3) Baseline branch scoring: `deterministic_netops_validator_v5` evaluates the `shared_initial_candidate`; baseline validator trace is in `execution/scoring/task-<id>-baseline.json` (`validation_before` and `validation_after` objects show `parsed_command_count`, `required_command_count`, `valid` flag, and `violation_codes` when present). 4) Guarded branch pre-processing and gating: a template-based input-sanitizer is applied (sanitizer logic and results are recorded in the task payload fields). The sanitized candidate is presented to the verifier-gate; if the sanitized candidate fails gating the policy allows at-most-one automated repair call. Repair-stage provider-call traces for episodes invoking repair are present under `execution/provider_calls` and referenced by `execution/scoring` artifacts (`repair_provider_trace_path` fields). The pipeline-level repair budget (≤ 1 repair call per episode) is enforced by the orchestration adapter (preregistration and execution manifests). Evidence: `stage_counts.repair = 6` and repair traces linked in `execution/scoring` files and in `analysis/deterministic_reconciliation.json`. 5) Final guarded validation: the validator re-evaluates the sanitized or repaired candidate, storing `validation_after` and `final_configuration` in `execution/scoring/task-<id>-guarded.json`. Paired artifacts for the same `shared_initial_candidate` include the `shared_initial_candidate_sha256` value present in both baseline and guarded scoring artifacts (e.g., `execution/scoring/task-`

`000001-baseline.json` and `execution/scoring/task-000001-guarded.json` share the same `shared_initial_candidate_sha256`).

Sanitizer and gating specifics (artifact-grounded). The input-sanitizer is template-driven: it enforces required slots, normalizes numeric fields (e.g., MTU integer normalization), and rejects or rewrites fields that violate per-task `allowed_fields/allowed_touched_fields` constraints; sanitized outputs are recorded in the task payload (`execution/task_manifest.jsonl` `task_payload.constraints` and `task_payload.intent`). The gating policy is deterministic: if `deterministic_netops_validator_v5` reports violations that fall into a policy-rejection set (e.g., protected-interface modification or missing required-step violations), the candidate is rejected; if violations are repairable under permitted-scope rules, the orchestrator may invoke a single repair generation attempt. These behaviors and the task-specific policies are visible in the task payload constraints and in per-episode validator traces (`execution/scoring/*` files).

Repair semantics and auditability. Repairs are restricted by preregistered policy: one additional LLM call per episode, limited to localized edits (slot corrections, sequence re-ordering) rather than free-form regeneration. Repair calls and resulting candidates are fully auditable: provider traces, `shared_initial_candidate_sha256`, `repair_prompt_sha256`, and `repair_provider_trace_path` appear in `execution/scoring` artifacts for repaired episodes (representative repaired episodes are traceable and correspond to the six repair-stage counts in `analysis/deterministic_reconciliation.json`). The six discordant `n10` cases observed in the paired contingency table correspond to episodes where a permitted repair converted an initially rejected draft into a validator-accepted final candidate; individual scoring artifacts show `validation_before` (invalid) and `validation_after` (valid) snapshots.

Outcome definitions, missingness handling, and exclusions. For each adversarial intent (paired unit) the binary outcome equals 1 if the deterministic validator would accept the final candidate for deployment and 0 otherwise. The preregistered `missingness_plan` excluded a paired unit only when both paired runs were technical failures (model API failure, verifier timeout, or parser error). Execution accounting: `planned_episode_count = 300`, `completed_episode_count = 282`, `failed_episode_count = 18`, `complete_pairs = 141` (`analysis/missingness_summary.csv`). Both-missing pairs = 9; provider-call audit: `hosted_model_call_event_count = 156`, `completed = 147`, `failed = 9` (`analysis/deterministic_reconciliation.json`). All exclusions and failure reasons are archived in `execution/execution_log.jsonl` and `analysis/missingness_summary.csv`.

Statistical-analysis procedures (artifact-backed). The preregistered primary test is the exact two-sided McNemar test on paired binary outcomes; contingency counts used by the test are archived at `analysis/paired_contingency_table.csv` (`n11 = 135`, `n10 = 6`, `n01 = 0`, `n00 = 0`). Paired effect estimation is presented as the absolute difference in success rates (guarded - baseline) with a bootstrap-percentile 95% CI obtained from 10,000 resamples (`bootstrap_seed = 1729`); bootstrap param-

eters and logs are archived in `analysis/analysis_log.jsonl`. Secondary analyses (exploratory) were planned for subgroup effects by mutation class (string-level, token-injection, slot-override) and for a benign-control false-positive-blocking rate; the benign-control ($M = 50$) was not executed in this run and therefore no empirical false-positive-blocking rates are reported.

Determinism, provenance, and reproducibility notes. All run artifacts required to reproduce the analysis are archived and referenced above (`execution/*` and `analysis/*`). The single protocol deviation (temperature = null in `execution/model_configuration.json`) is disclosed; because the pipeline used `shared_initial_candidate` semantics and provider-call accounting shows a single shared generation per pair (`stage_counts.shared_initial_generation = 150`), paired comparability is preserved for this run. For deterministic reproduction we recommend explicitly setting temperature = 0 and re-running the archived `execution/task_manifest.jsonl` with identical orchestration to produce bit-identical provider traces under the same provider semantics. Representative artifact paths for forensic verification: `execution/task_manifest.jsonl`, `execution/responses/task-000001-shared-initial.txt`, `execution/scoring/task-000001-baseline.json`, `execution/scoring/task-000001-guarded.json`, `analysis/deterministic_reconciliation.json`, `analysis/paired_contingency_table.csv`, `analysis/analysis_log.jsonl`.

Pre-registered safeguards and contamination controls. The preregistration included contamination controls (artifact hashing and embargo rules for hazardous artifacts) and explicit exclusion rules for sanitized benign intents with excessive slot-drop (>0.3). Contamination summaries and integrity checks are archived in `analysis/contamination_summary.csv` and the `execution/execution_manifest.json`. Any artifact-flagging and masking procedures applied during archival are recorded; flagged hazardous raw artifacts are embargoed per the pre-registered contamination policy.

Summary of methodological guarantees and limitations. Methodologically, the design enforces a paired comparison isolating downstream guarded-pipeline effects by reusing a single `shared_initial_candidate` per pair and limiting repairs to a single bounded attempt. Auditability is provided by per-episode validator traces, provider-call traces, and deterministic reconciliation artifacts. Limitations inherent in the design (single-model, synthetic tasks, validator coverage, and the protocol deviation noted above) are described in Threats to Validity and Limitations; these do not alter the integrity of the reported artifact-grounded paired comparison but bound external generalization.

IV. RESULTS

Execution and counts. The frozen run produced planned episodes = 300, completed episodes = 282, failed episodes = 18, and model provider calls used = 156 (`execution/execution_manifest.json`). The primary confirmatory analysis used 141 complete pairs (`analysis/missingness_summary.csv`). Condition summary

(`analysis/condition_summary.csv`) shows `baseline_successes = 135/141` (0.9574468085) and `guarded_successes = 141/141` (1.0).

Paired contingency and inference. The archived paired contingency (`analysis/paired_contingency_table.csv`) is $n_{11} = 135$, $n_{10} = 6$ (`guarded=1`, `baseline=0`), $n_{01} = 0$, $n_{00} = 0$. An exact two-sided McNemar test on the observed paired outcomes yields $p = 0.03125$. The point estimate (`guarded - baseline`) = 0.04255319148936165. The 95% bootstrap percentile CI (10,000 resamples, seed = 1729) is [0.014184397163120567, 0.07801418439716312]; bootstrap parameters and log entries are archived in `analysis/analysis_log.jsonl`.

Discordant-case diagnostics. The six discordant n_{10} pairs are fully traceable in the archive and are associated with repair-stage activity: `analysis/deterministic_reconciliation.json` records repair = 6 and the corresponding per-episode scoring artifacts contain validator before/after snapshots showing sanitizer and/or permitted repair changes that converted a rejected initial draft into an accepted final candidate. Representative artifacts include `execution/responses/task-000001-shared-initial.txt`, `execution/scoring/task-000001-baseline.json` and `execution/scoring/task-000001-guarded.json`; these artifacts explicitly show the identical `shared_initial_candidate` referenced by both scoring artifacts and the validator’s `validation_before` and `validation_after` objects.

Missingness sensitivity and unexecuted items. Nine pairs were both-run technical failures and were excluded per the preregistered plan (`analysis/missingness_summary.csv`: `both_missing_pairs = 9`). Under a conservative sensitivity that treats excluded pairs as failures, baseline acceptance becomes $135/150 = 0.9000$ and guarded acceptance $141/150 = 0.9400$. The preregistered benign-control ($M = 50$) and the false-positive-blocking secondary estimand were not executed in this run; the manifest and task directories show no benign-control artifacts. Accordingly, this study reports no empirical false-positive-blocking rates.

V. DISCUSSION

Hypothesis reconciliation and interpretive boundary. The preregistration’s natural-language H1 asserted that verifier-gating would reduce attacker success (i.e., fewer unsafe configurations accepted). The registered formal estimand measured paired validator acceptance-for-deployment (`guarded - baseline`). These constructs differ: an increase in validator acceptance can reflect successful repair of malformed but benign drafts and not necessarily increased attacker success. Our observed guarded→higher acceptance (≈ 4.255 percentage points) occurred because sanitizer and permitted repair recovered repairable drafts that the baseline validator rejected. Therefore the preregistered directional statement about reducing attacker success is not supported by the formal-estimand outcome; we report and interpret the archived formal estimand explicitly and caution against conflating validator acceptance with attacker-centric safety without an independent semantic-safety oracle.

Operational implications and repair-gate co-design. Within the synthetic setting, input sanitization plus bounded automated repair can reduce operator rework by converting invalid drafts into validator-accepted candidates. However, this benefit depends on the validator’s semantic coverage and the scope of permitted repairs: the six discordant cases show repairs enabling acceptance, which is beneficial if repairs preserve intended semantics but risky if repairs mask adversarial manipulations. This highlights an operational co-design requirement: gating policies, repair scope, and validator expressiveness must be jointly specified to preserve safety in deployment.

Practical trade-offs illustrated by the archive. The artifact evidence (analysis/deterministic_reconciliation.json, analysis/paired_contingency_table.csv, and per-episode scoring artifacts such as execution/scoring/task-000001-guarded.json) shows precisely how small numbers of permitted repairs materially changed final acceptance. That pattern exposes a predictable trade-off: stricter gating (reject-on-violation) reduces the chance of a repaired but semantically altered candidate being accepted, at the cost of more operator interventions; more permissive policies with bounded repairs reduce operator workload but raise the need for stronger semantic checks or human review on repaired cases. These trade-offs are operational, measurable with the archived logs (repair_stage_count = 6, hosted_model_call_event_count = 156) and must be chosen to reflect acceptable risk budgets in deployment.

Failure modes of generate-validate-repair loops. The archived validator traces document three related failure modes relevant to operational design: (1) reparable syntactic/ordering errors where repair restores intent-preserving structure (the desirable class observed for the six discordants); (2) repair that fills missing fields or normalizes values in ways that change semantics subtly (requires semantic-safety or reachability checks beyond the current validator); and (3) cases of technical failure or timeouts (18/300 episodes, both-run failures = 9) where neither repair nor gating completed, reducing effective sample size (analysis/missingness_summary.csv). Operational pipelines should instrument and surface these distinct failure categories to operators and governance audits using the same per-episode trace primitives present in our archive.

Reproducibility and the protocol deviation. The unset runtime sampling temperature (temperature = null in execution/model_configuration.json) is a recorded protocol deviation. Its practical implication depends on provider default behavior; because the run used shared_initial_candidate semantics (stage_counts.shared_initial_generation = 150) and provider-call accounting confirms the observed call pattern (analysis/deterministic_reconciliation.json), paired comparability for the formal estimand is preserved in this execution. Nevertheless, for deterministic reproduction we recommend setting temperature = 0 explicitly (execution/model_configuration.json) and re-running the archived task manifest; the archive provides all artifacts necessary to compare outcomes deterministically (execution/responses/*, execution/scoring/*, analysis/*).

Governance, auditing, and measurement recommendations

grounded in archived evidence. The run demonstrates the value of (a) detailed per-episode validator traces (validation_before/after objects in scoring artifacts) to audit repairs; (b) explicit repair audit logs and provenance (present for repaired episodes) so that operator review can target only changed cases; and (c) a preregistered benign-control to quantify false-positive-blocking (not executed here). The archive’s contamination and missingness summaries (analysis/contamination_summary.csv is empty; analysis/missingness_summary.csv reports counts) show these governance metrics can be computed automatically from the same instrumentation used in this study.

Limits to operational generalization. The discussion above is tethered to the archive: a single LLM (declared gpt-5-mini), a single deterministic validator implementation, and synthetic templates. These constraints mean the measured acceptance-rate change is a within-bundle operational observation, not a claim of cross-vendor or production robustness. The ceiling effect (high baseline acceptance) reduces practical headroom for improvement; this is visible in analysis/condition_summary.csv and motivates future runs that (1) execute the preregistered benign-control to measure false positives, (2) adopt attacker-centric semantic-safety or reachability oracles in addition to syntactic validators, and (3) diversify model and validator families before making operational deployment claims.

Concluding interpretive note. The archived data concretely show that bounded verifier-gated repair can increase validator acceptance by recovering reparable drafts; whether that outcome improves or harms security depends on whether repairs preserve intended semantics—a property not fully covered by the present deterministic validator and therefore not directly measured here. The empirical artifacts (per-episode scoring traces, repair counts, and paired contingency outputs) provide the necessary operational observables to design follow-up confirmations that target attacker-centric estimands and benign-control measurements.

VI. LIMITATIONS

- Synthetic task bank and intent templates — not real production traffic or operator logs; results may not generalize to field datasets.
- Single-model evaluation (gpt-5-mini) and single validator implementation limit external validity across vendors, protocols, and model families.
- Validator coverage constrained to encoded safety properties (CLI grammar, required sequences, protected-interface invariants); some protocol-level or dynamic runtime semantics are not represented.
- Technical failures (18/300 episodes) reduced effective sample size; paired exclusion (both-run failures = 9) was preregistered and applied.
- Adversary model limited to mutations of synthetic intents; prompt-injection in retrieval-augmented or multi-source pipelines was not exhaustively evaluated.

- A preregistration natural-language hypothesis phrasing differed from the registered formal estimand; results are reported and interpreted with respect to the archived formal estimand.
- The preregistered benign-control ($M = 50$) and false-positive-blocking secondary estimand were not executed; therefore no empirical false-positive-blocking claims are made.

VII. CONCLUSION

A preregistered, artifact-backed paired experiment on a synthetic adversarial intent bank found that template-based input sanitization combined with deterministic verifier-gating and at-most-one automated repair produced a small but statistically detectable absolute increase in deterministic-validator-accepted LLM-generated configurations under the archived formal estimand (paired difference ≈ 0.04255 ; $p = 0.03125$; 95% CI [0.01418, 0.07801]). Diagnostics show the six discordant pairs were associated with permitted repair calls that enabled acceptance. Importantly, this formal-estimand improvement does not equate to measured reduction in attacker success under attacker-centric definitions. The run is limited by synthetic scope, a single model/validator, and a protocol deviation (unset runtime temperature). Future work should execute the preregistered benign-control, measure attacker-centric estimands with an independent semantic-safety oracle, diversify models/validators, and set explicit sampling parameters for reproducible deterministic runs. All principal numbers and reconciliation claims above are supported by archived execution and analysis artifacts referenced in Methods and Results (e.g., `execution/task_manifest.jsonl`; `execution/responses/task-000001-shared-initial.txt`; `execution/scoring/task-000001-baseline.json`; `execution/scoring/task-000001-guarded.json`; `analysis/paired_contingency_table.csv`; `analysis/condition_summary.csv`).

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: a human Research Director selected the topic family (Generative AI / LLMs for NetOps) and constrained the initial master prompt prior to the autonomy lock; all literature review, preregistration, study selection, code generation, experiment execution, analysis, peer-review cycles, manuscript drafting, and revision reported here were performed autonomously after the lock. Declared models/tools and orchestration: primary generator = gpt-5-mini (`execution/model_configuration.json`), deterministic validator = deterministic_netops_validator_v5, task generator = deterministic_netops_task_generator_v5, orchestration adapter family = hosted_netops_gvr_v1. Preregistration: study cand-2026-03 was preregistered and archived prior to observing outcomes (preregistration_sha256: ba6ed25e

84c7240fbe94f2d50b3e778fbfffd810ff2db632df8946e85f0f1373). Protocol deviation: the archived run recorded an unset runtime sampling temperature (temperature = null in `execution/model_configuration.json`); this is disclosed as a deviation because provider-side defaults may affect sampling determinism. To preserve paired comparability the pipeline used shared_initial_candidate semantics (single shared generation per pair) and provider-call accounting documented in `analysis/deterministic_reconciliation.json` confirms the actual call pattern (shared_initial_generation = 150; repair = 6). Key archived artifact references (examples): `execution/task_manifest.jsonl`; `execution/responses/task-000001-shared-initial.txt`; `execution/scoring/task-000001-baseline.json`; `execution/scoring/task-000001-guarded.json`; `analysis/deterministic_reconciliation.json`; `analysis/paired_contingency_table.csv`; `analysis/condition_summary.csv`; `analysis/analysis_log.jsonl`; `execution/model_configuration.json`; `execution/execution_manifest.json`. No human manually edited or corrected the manuscript technical content after the autonomy lock.

REFERENCES

- [1] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>
- [2] <https://doi.org/10.1145/2342441.2342452>
- [3] <https://doi.org/10.1145/3626111.3628194>
- [4] <https://doi.org/10.1145/3605764.3623985>
- [5] <http://arxiv.org/abs/2302.04761>
- [6] <https://doi.org/10.1145/3656296>
- [7] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447
- [8] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
- [9] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, Mario Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", 2023, 10.1145/3605764.3623985
- [11] Yupeng Chang, Xu Wang, Jindong Wang, Yuan-Hsuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, Xing Xie, "A Survey on Evaluation of Large Language Models", 2023, 10.48550/arxiv.2307.03109
- [12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Răileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools", 2023, 10.48550/arxiv.2302.04761
- [13] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [14] Peyman Kazemian, George Varghese, Nick McKeown, "Header space analysis: static checking for networks", 2012